

Instituto Politécnico Nacional

Escuela Superior de Cómputo



Cryptography

Session 4

Secret-key cryptography

Autor:
Randy Villanueva Guzmán

Profesora:
Sandra Díaz

Grupo:
3CM9

April 26, 2020

Contents

1 Module pyCryptodome **3**

2 Pseudorandom generator **3**

3 Key generation for secret-key **4**

4 Encryption and decryption using a stream cipher **4**
4.1 ChaCha20 and XChaCha20 4
4.2 Salsa20 5

5 Encryption and decryption using a block ciphers **6**
5.1 AES 6
5.2 3DES 7
5.3 RC2 8

6 Modes of operation **9**
6.1 ECB 10
6.2 CBC 10
6.3 CTR 11
6.4 CFB 12
6.5 OFB 12

7 Encryption and decryption combining a block cipher and each mode of operation. **13**
7.1 Plaintext for CBC and CTR 13
7.2 CBC 14
7.3 CTR 17
7.4 Plaintext for CFB and OFB 18
7.5 CFB 18
7.6 OFB 20

1 Module pyCryptodome

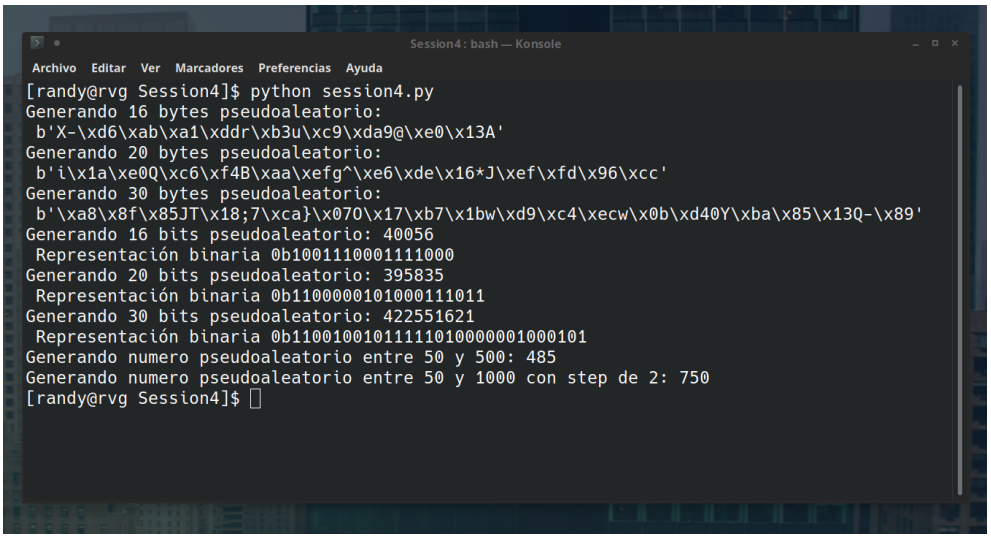
pyCryptodome es un módulo autónomo en python con primitivas criptográficas de bajo nivel. Considero que es una buena opción ya que es la mejora de otros módulos, está programado en python puro a excepción de los cifradores de bloque, donde se implementa extensiones en C para un mejor rendimiento. A diferencia de otros módulos, este implementa funciones para el manejo de padding y la generación de bytes pseudoaleatorias

2 Pseudorandom generator

Para generar números y bits de manera pseudoaleatoria *pyCryptodome* ofrece las siguientes funciones:

- get_random_bytes$N$
- random.getrandbits(N)
- random.randrange([start,]stop[, step])
- random.randint(a, b)
- random.choice(seq)
- random.shuffle(seq)
- random.sample(population, k)

Con estas funciones se pueden generar bits, numeros enteros, numeros en un rango y seleccionar dentro de una secuencia de numeros.




```
#Random number generation
random_16_bytes = get_random_bytes( 16 )
random_20_bytes = get_random_bytes( 20 )
random_30_bytes = get_random_bytes( 30 )
print( 'Generando 16 bytes pseudoaleatorio:\n {}' .format( random_16_bytes) )
print( 'Generando 20 bytes pseudoaleatorio:\n {}' .format( random_20_bytes) )
print( 'Generando 30 bytes pseudoaleatorio:\n {}' .format( random_30_bytes) )

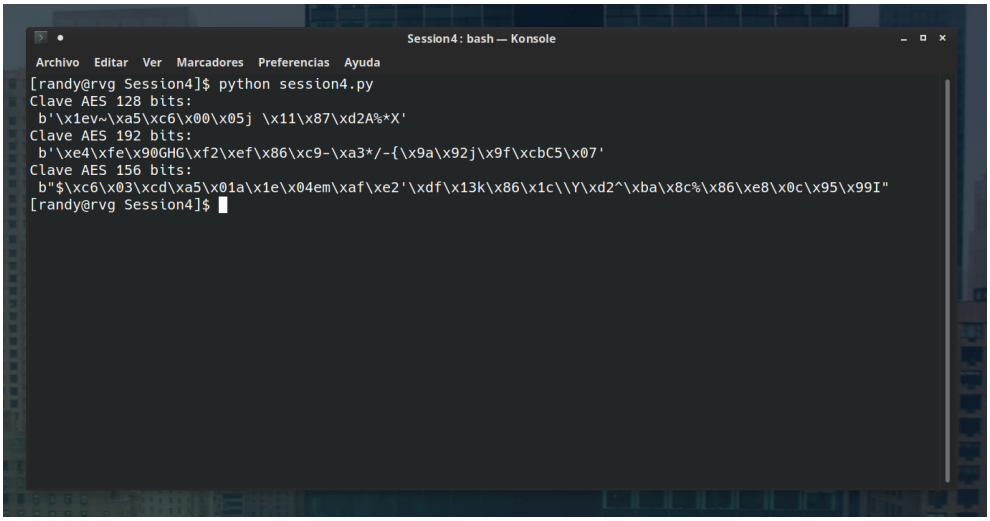
random_16_bits = getrandbits( 16 )
random_20_bits = getrandbits( 20 )
random_30_bits = getrandbits( 30 )
print( 'Generando 16 bits pseudoaleatorio: {}\n Representación binaria {}'
```

```
        .format( random_16_bits , bin( random_16_bits ) ) )
print( 'Generando 20 bits pseudoaleatorio: {}\n Representación binaria {}'
        .format( random_20_bits , bin( random_20_bits ) ) )
print( 'Generando 30 bits pseudoaleatorio: {}\n Representación binaria {}'
        .format( random_30_bits , bin( random_30_bits ) ) )

rand_int = randint( 50 , 500 )
rand_range = randrange( 50 , 1000 , 2 )
print( 'Generando numero pseudoaleatorio entre 50 y 500: {}' .format(rand_int) )
print( 'Generando numero pseudoaleatorio entre 50 y 1000 con step de 2: {}' .format(rand_range) )
```

3 Key generation for secret-key

Para la generación de llaves se puede utilizar la función *get_random_bytes()* para obtener una llave de acuerdo al tamaño que necesitemos, por ejemplo para AES de 128,192, 256 o para DES de 168 y 112.



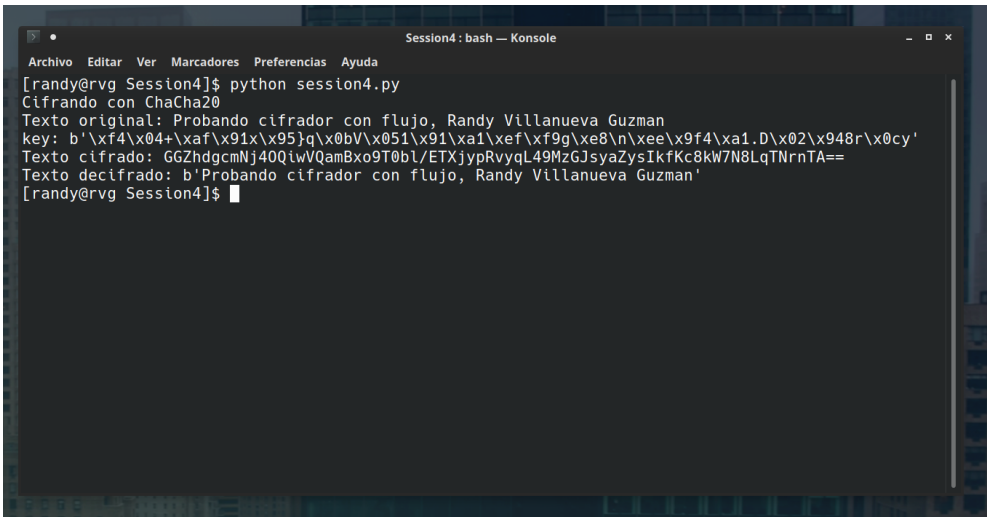

```
key_Aes_128 = get_random_bytes( 16 )
key_Aes_192 = get_random_bytes( 24 )
key_Aes_256 = get_random_bytes( 32 )

print( 'Clave AES 128 bits:\n {} ' .format( key_Aes_128 ) )
print( 'Clave AES 192 bits:\n {} ' .format( key_Aes_192 ) )
print( 'Clave AES 156 bits:\n {} ' .format( key_Aes_256 ) )
```

4 Encryption and decryption using a stream cipher

4.1 ChaCha20 and XChaCha20

Es un cifrador de flujo diseñado por By Daniel J. Bernstein. La clave que utiliza es de un tamaño de 256 bits (32 bytes) y la misma clave no debe ser reutilizada en diferentes cifrados. Existen 3 variantes con clave de 8, 12 y 24 bytes.
Los textos simples y los textos cifrados (entrada / salida) solo pueden ser bytes, bytearray o memoryview. En Python 3, no puedes pasar cadenas. En Python 2, no puede pasar cadenas Unicode.




```
#ChaCha20
#encrypt data
plaintext = b'Probando cifrador con flujo, Randy Villanueva Guzman'
key = get_random_bytes( 32 )
cipher = ChaCha20.new( key=key )
ciphertext = cipher.encrypt( plaintext )

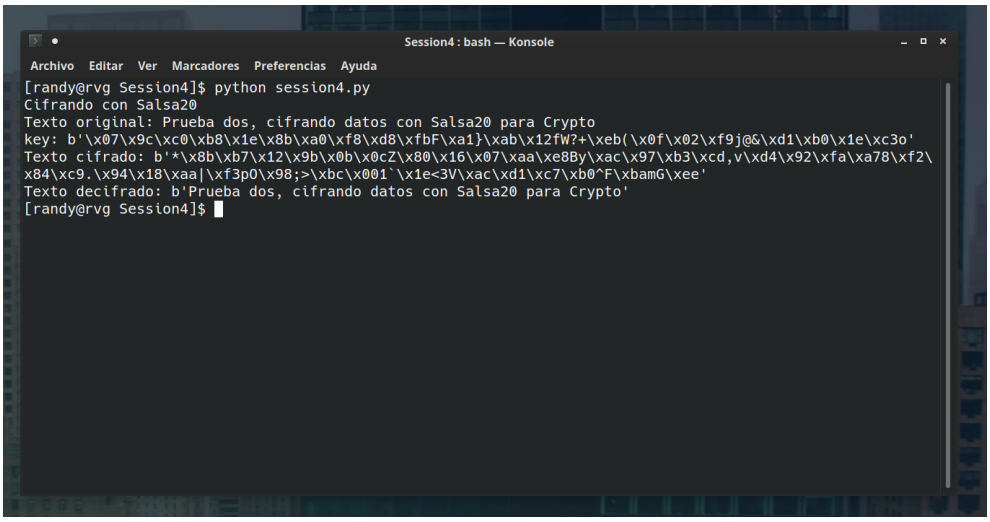
nonce = b64encode(cipher.nonce).decode('utf-8')
ct = b64encode(ciphertext).decode('utf-8')
result = json.dumps({'nonce':nonce, 'ciphertext':ct})

print( 'Cifrando con ChaCha20' )
print( 'Texto original: {}' .format(plaintext.decode('UTF-8')) )
print( 'key: {}' .format(key) )
print( 'Texto cifrado: {}' .format(ct) )

#decrypt data
try:
    b64 = json.loads( result )
    nonce = b64decode( b64['nonce'] )
    ciphertext = b64decode( b64['ciphertext'] )
    decipher = ChaCha20.new( key=key, nonce=nonce )
    plaintext = decipher.decrypt( ciphertext )
    print( 'Texto decifrado: {}' .format(plaintext) )
except:
    print( 'Incorrect decryption' )
```

4.2 Salsa20

Cifrador de flujo diseñado por Daniel J. Bernstein. La clave de preferencia es de 256 bits, pero tambien puede trabajar con una de 128 bits.




```
#Salsa20
#encrypt data
plaintext = b'Prueba dos, cifrando datos con Salsa20 para Crypto'
secret = get_random_bytes( 32 )
cipher = Salsa20.new( key=secret )
msg = cipher.nonce + cipher.encrypt(plaintext)

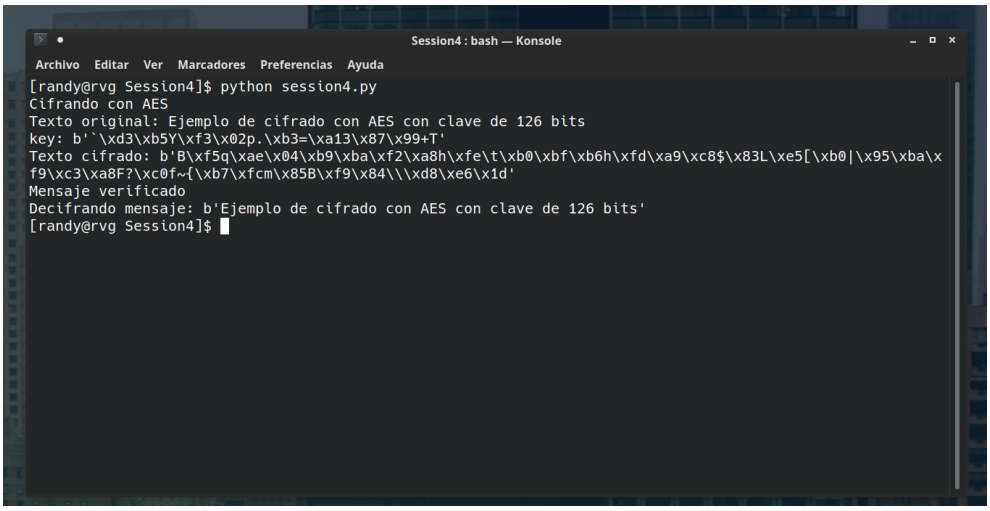
print( 'Cifrando con Salsa20' )
print( 'Texto original: {}' .format(plaintext.decode('UTF-8')) )
print( 'key: {}' .format(secret) )
print( 'Texto cifrado: {}' .format(msg) )

#decrypt data
try:
    msg_nonce = msg[:8]
    ciphertext = msg[8:]
    cipher = Salsa20.new( key=secret, nonce=msg_nonce )
    plaintext = cipher.decrypt(ciphertext)
    print( 'Texto decifrado: {}' .format(plaintext) )
except:
    print( 'Incorrect decryption' )
```

5 Encryption and decryption using a block ciphers

5.1 AES

Es un cifrador de bloque estándar por NIST. Tiene un tamaño de bloque de datos fijo de 16 bytes. Sus claves pueden ser de 128, 192 o 256 bits de largo.



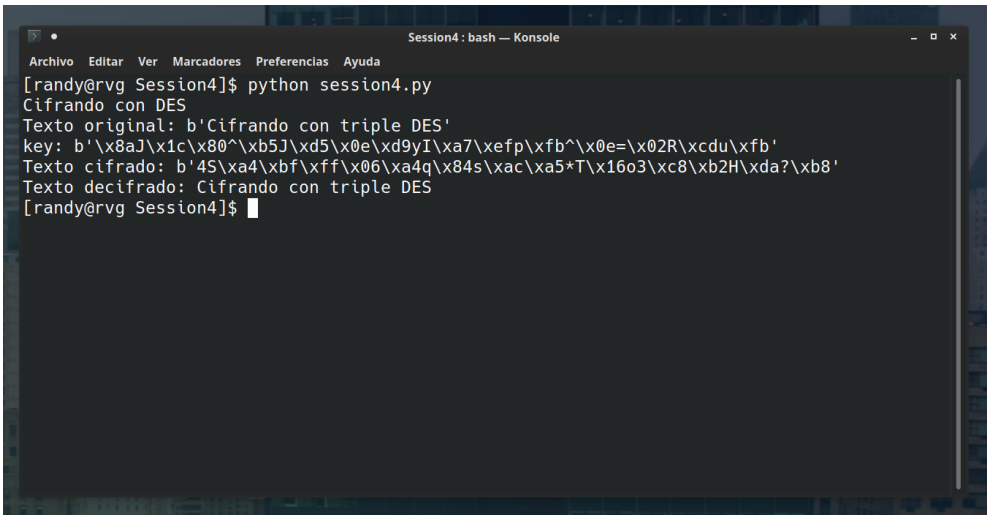

```python
#AES
key = get_random_bytes( 16 )
data = b'Ejemplo de cifrado con AES con clave de 126 bits'
cipher = AES.new(key, AES.MODE_EAX)
nonce = cipher.nonce
ciphertext, tag = cipher.encrypt_and_digest( data )

print( 'Cifrando con AES' )
print( 'Texto original: {}' .format(data.decode('UTF-8')) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( ciphertext ) )

decipher = AES.new(key, AES.MODE_EAX, nonce=nonce)
plaintext = decipher.decrypt(ciphertext)
try:
    decipher.verify( tag )
    print( 'Mensaje verificado' )
    print( 'Decifrando mensaje: {}' .format( plaintext ) )
except:
    print( 'Key incorrect or message corrupted' )
```

5.2 3DES

Triple DES (o TDES o TDEA o 3DES) es un cifrado de bloque simétrico estandarizado por NIST en SP 800-67 Rev1. TDES tiene un tamaño de bloque de datos fijo de 8 bytes. Consiste en la cascada de 3 cifrados DES únicos (EDE: cifrado - descifrado - cifrado), donde cada etapa utiliza una subclave DES independiente.




```
#3DES

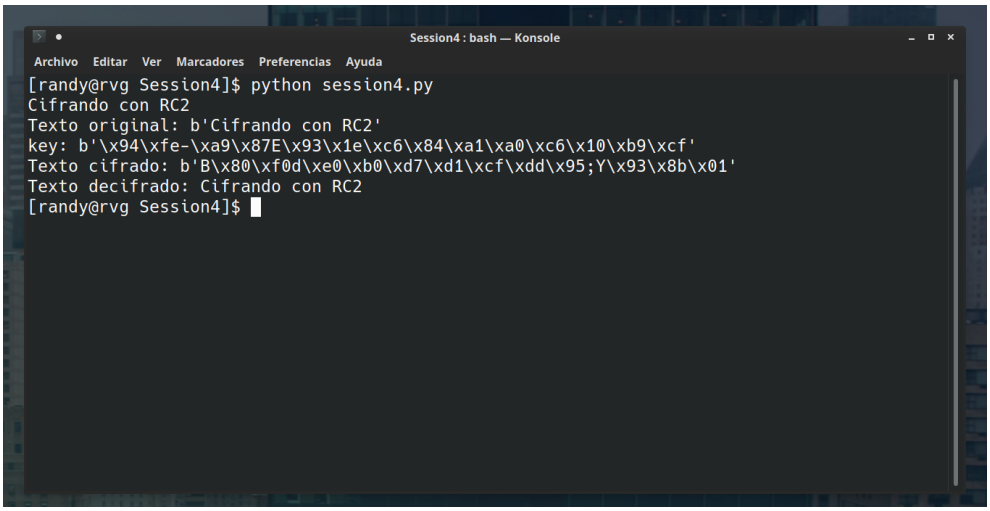
while True:
    try:
        key_3_Des = DES3.adjust_key_parity( get_random_bytes(24) )
        break
    except ValueError:
        pass

#cipher
plaintext = b'Cifrando con triple DES '
iv = get_random_bytes(8)
cipher = DES3.new( key=key_3_Des , mode=DES3.MODE_CFB , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key_3_Des ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key_3_Des , mode=DES3.MODE_CFB , iv=iv)
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )
```

5.3 RC2

RC2 (Cifrado de Rivest, versión 2) es un cifrado de bloque simétrico diseñado por Ron Rivest en 1987.RC2 tiene un tamaño de bloque de datos fijo de 8 bytes. La longitud de sus claves puede variar de 8 a 128 bits. Aunque RC2 no está roto criptográficamente, no se ha analizado tan a fondo cómo AES, que también es más rápido que RC2.



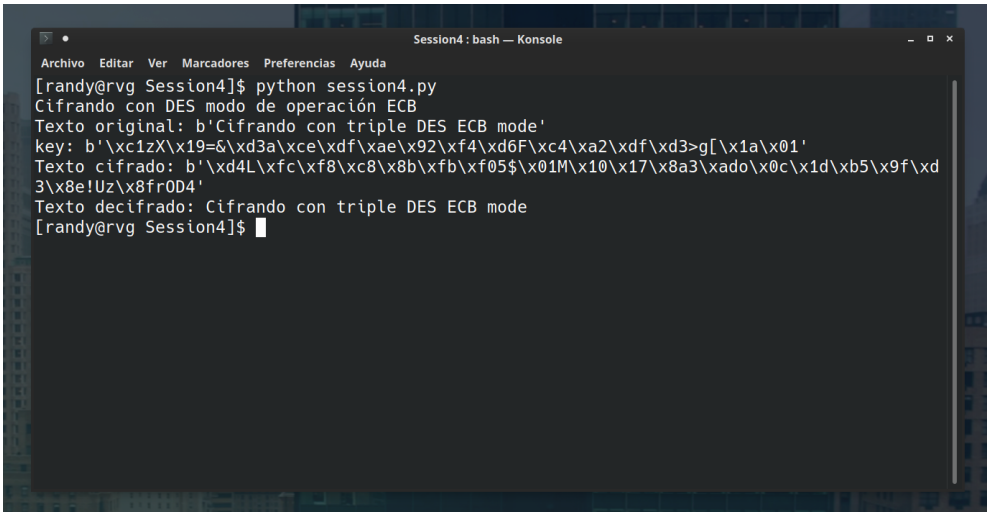

```
#RC2
plaintext = b'Cifrando con RC2'
key = get_random_bytes( 16 )
iv = get_random_bytes( 8 )
cipher = ARC2.new( key=key , mode=ARC2.MODE_CFB , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con RC2' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )
decipher = ARC2.new( key=key, mode=ARC2.MODE_CFB , iv=iv )
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )
```

6 Modes of operation

Para esta sección se genrara una llave y se usara en todos los modos de operación. La llave se genenera de la siguiente manera:

```
while True:
    try:
        key = DES3.adjust_key_parity( get_random_bytes(24) )
        break
    except ValueError:
        pass
```

6.1 ECB

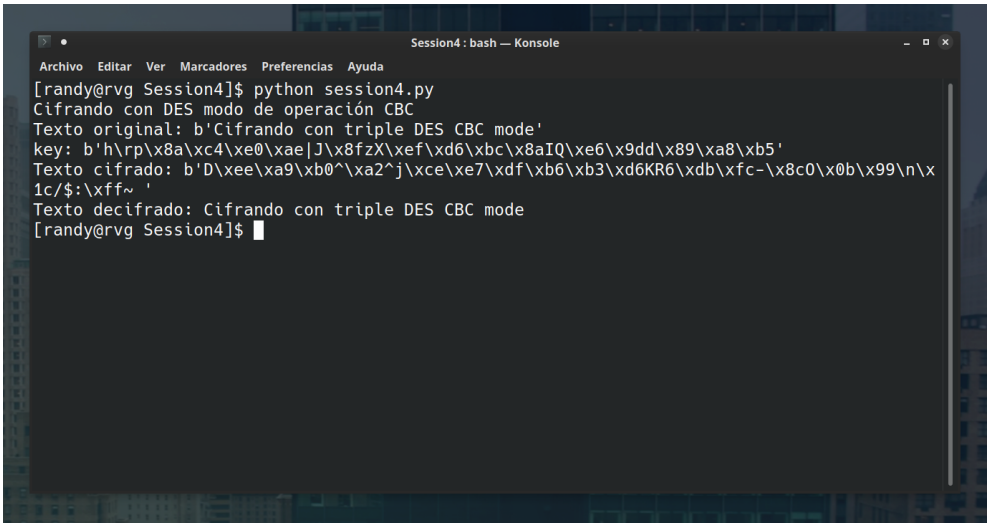



```
#ECB mode

plaintext = b'Cifrando con triple DES ECB mode'
cipher = DES3.new( key=key , mode=DES3.MODE_ECB  )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación ECB' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_ECB )
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )
```

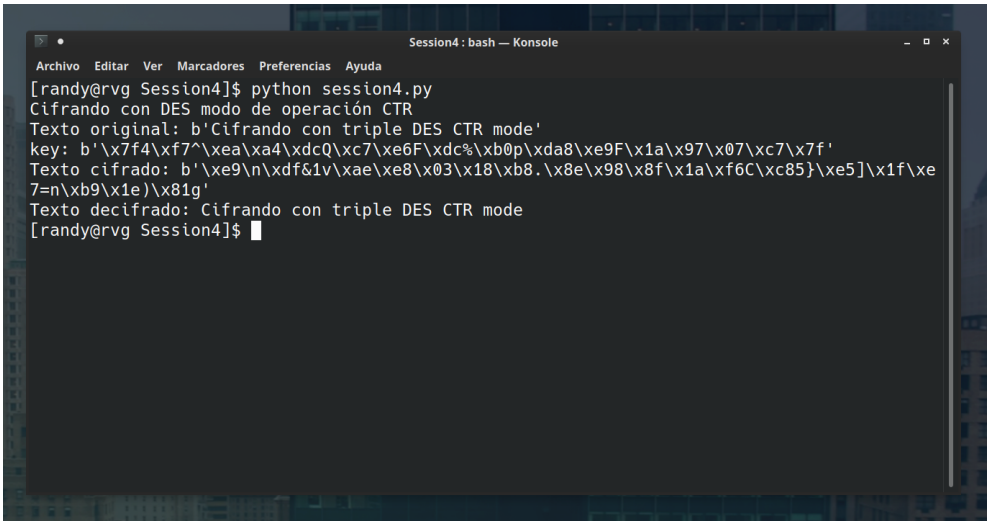
6.2 CBC



```
#CBC mode
plaintext = b'Cifrando con triple DES CBC mode'
iv = get_random_bytes(8)
cipher = DES3.new( key=key , mode=DES3.MODE_CBC , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación CBC' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_CBC , iv=iv )
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )
```

6.3 CTR

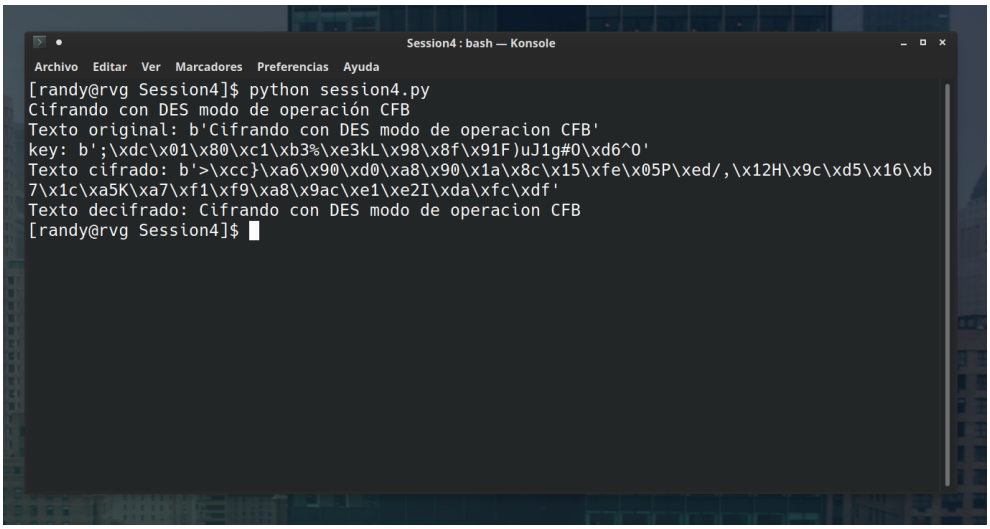



```
#CTR mode

#cipher
plaintext = b'Cifrando con triple DES CTR mode'
nonce = Random.new().read(int(DES3.block_size/2))
ctr = Counter.new(DES3.block_size*4, prefix=nonce)
cipher = DES3.new( key=key , mode=DES3.MODE_CTR , counter=ctr )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación CTR' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_CTR , counter=ctr )
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )
```

6.4 CFB

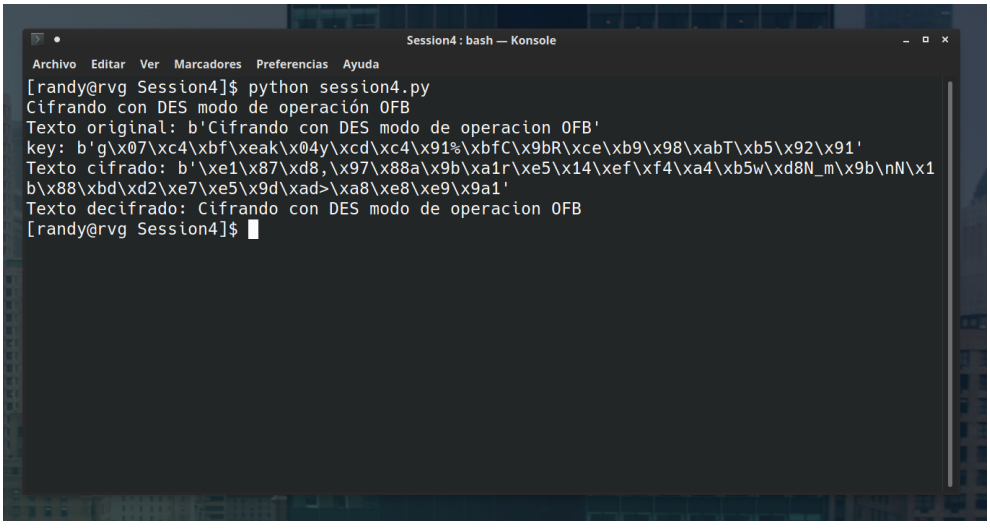



```
#CFB mode

#cipher
plaintext = b'Cifrando con DES modo de operacion CFB'
iv = get_random_bytes(8)
cipher = DES3.new( key=key , mode=DES3.MODE_CFB , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación CFB' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_CFB , iv=iv)
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )
```

6.5 OFB



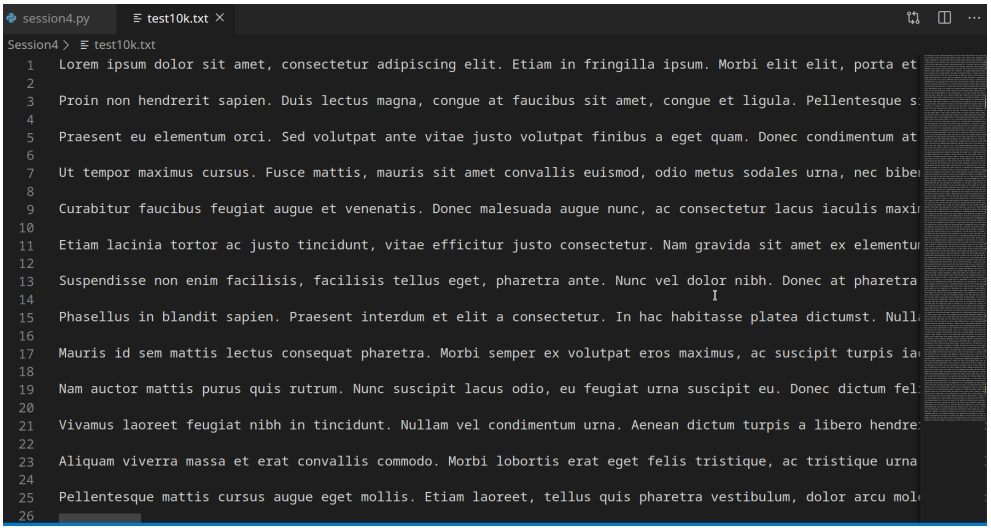
```
#OFB mode

#cipher
plaintext = b'Cifrando con DES modo de operacion OFB'
iv = get_random_bytes(8)
cipher = DES3.new( key=key , mode=DES3.MODE_OFB , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación OFB' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

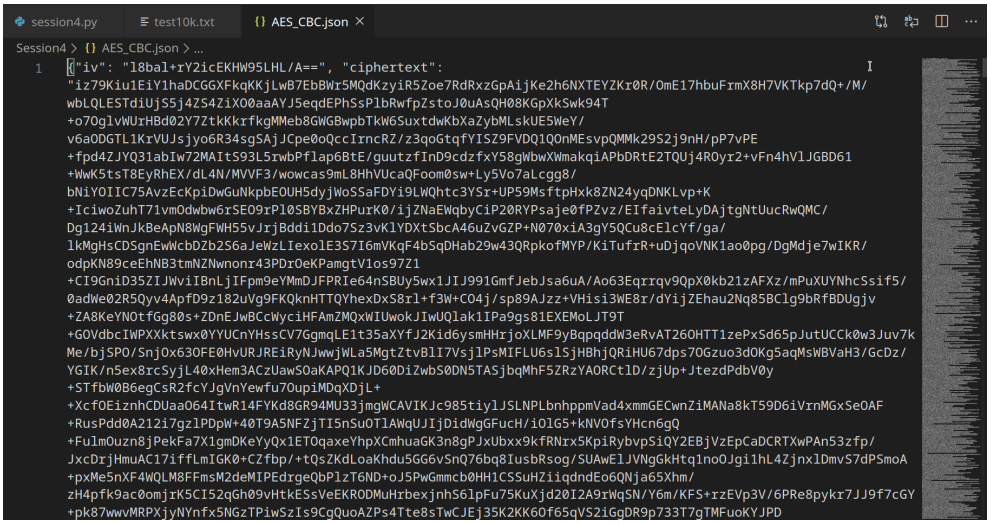
#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_OFB , iv=iv)
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )
```

7 Encryption and decryption combining a block cipher and each mode of operation.

7.1 Plaintext for CBC and CTR



7.2 CBC






```python
#modes of operation

while True:
    try:
        key = DES3.adjust_key_parity( get_random_bytes(24) )
        break
    except ValueError:
        pass

#ECB mode

plaintext = b'Cifrando con triple DES ECB mode'
cipher = DES3.new( key=key , mode=DES3.MODE_ECB  )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación ECB' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
```

```
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_ECB )
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )

#CBC mode
plaintext = b'Cifrando con triple DES CBC mode'
iv = get_random_bytes(8)
cipher = DES3.new( key=key , mode=DES3.MODE_CBC , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación CBC' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_CBC , iv=iv )
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )

#CTR mode

#cipher
plaintext = b'Cifrando con triple DES CTR mode'
nonce = Random.new().read(int(DES3.block_size/2))
ctr = Counter.new(DES3.block_size*4, prefix=nonce)
cipher = DES3.new( key=key , mode=DES3.MODE_CTR , counter=ctr )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación CTR' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_CTR , counter=ctr )
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )

#CFB mode

#cipher
plaintext = b'Cifrando con DES modo de operacion CFB'
iv = get_random_bytes(8)
cipher = DES3.new( key=key , mode=DES3.MODE_CFB , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación CFB' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_CFB , iv=iv)
decrypt_msg = decipher.decrypt( msg )
```

```
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )

#OFB mode

#cipher
plaintext = b'Cifrando con DES modo de operacion OFB'
iv = get_random_bytes(8)
cipher = DES3.new( key=key , mode=DES3.MODE_OFB , iv=iv )
msg = cipher.encrypt( plaintext )
print( 'Cifrando con DES modo de operación OFB' )
print( 'Texto original: {}' .format( plaintext ) )
print( 'key: {}' .format( key ) )
print( 'Texto cifrado: {}' .format( msg  ) )

#decipher
decipher = DES3.new( key=key , mode=DES3.MODE_OFB , iv=iv)
decrypt_msg = decipher.decrypt( msg )
print( 'Texto decifrado: {} ' .format( decrypt_msg.decode('UTF-8') ) )

file = open( 'test10k.txt' )
text = file.read()
file.close()

text = bytes( text,"UTF-8" )

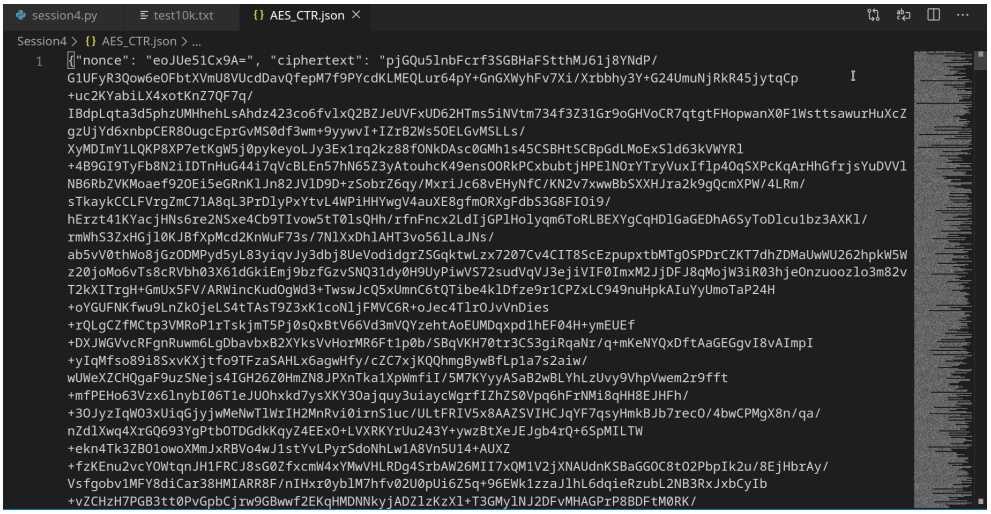
#CBC

key = get_random_bytes( 16 )
cipher = AES.new(key, AES.MODE_CBC)
ct_bytes = cipher.encrypt( pad( text, AES.block_size ) )
iv = b64encode(cipher.iv).decode('utf-8')
ct = b64encode(ct_bytes).decode('utf-8')
result = json.dumps({'iv':iv, 'ciphertext':ct})
file = open('AES_CBC.json','w')
file.write( result )
file.close()

file = open( 'AES_CBC.json','r' )
json_input = file.read()
file.close()

try:
    b64 = json.loads(json_input)
    iv = b64decode(b64['iv'])
    ct = b64decode(b64['ciphertext'])
    cipher = AES.new(key, AES.MODE_CBC, iv)
    pt = unpad(cipher.decrypt(ct), AES.block_size)
    file = open('Decrypt_AES_CBC.txt','w')
    file.write( pt.decode('UTF-8') )
    file.close()
except:
    print('Incorrect decryption CBC')
```

7.3 CTR






```python
#cipher
key = get_random_bytes(16)
cipher = AES.new(key, AES.MODE_CTR)
ct_bytes = cipher.encrypt( text )
nonce = b64encode(cipher.nonce).decode('utf-8')
ct = b64encode(ct_bytes).decode('utf-8')
result = json.dumps({'nonce':nonce, 'ciphertext':ct})

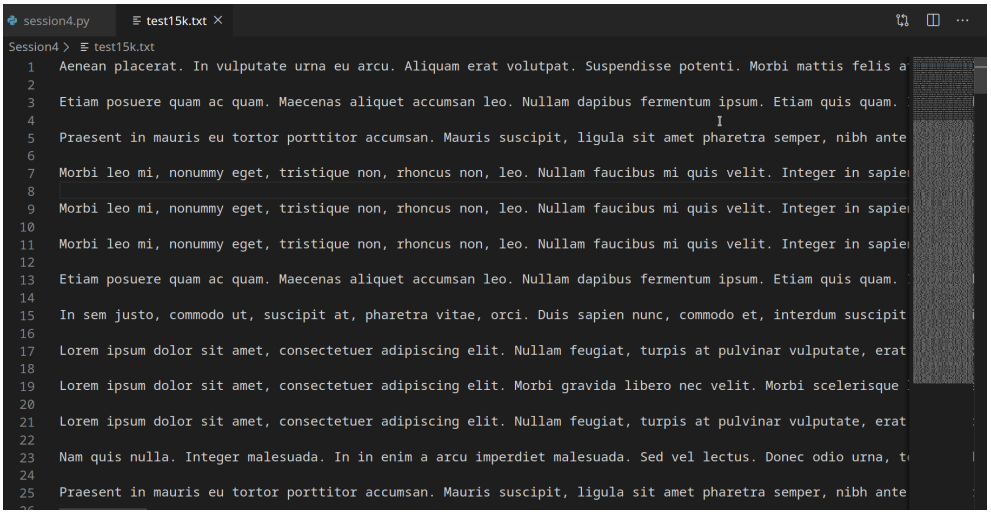
file = open('AES_CTR.json','w')
file.write( result )
file.close()

file = open( 'AES_CTR.json','r' )
json_input = file.read()
file.close()

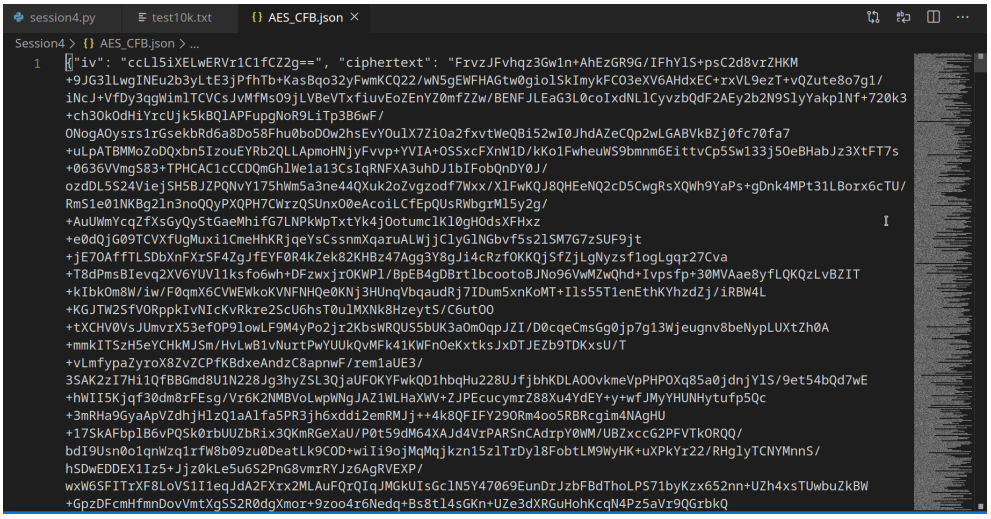
#decipher
try:
```

```
    b64 = json.loads(json_input)
    nonce = b64decode(b64['nonce'])
    ct = b64decode(b64['ciphertext'])
    cipher = AES.new(key, AES.MODE_CTR, nonce=nonce)
    pt = cipher.decrypt(ct)
    file = open('Decrypt_AES_CTR.txt','w')
    file.write( pt.decode('UTF-8') )
    file.close()
except:
    print('Incorrect decryption CTR')
```

7.4 Plaintext for CFB and OFB



7.5 CFB





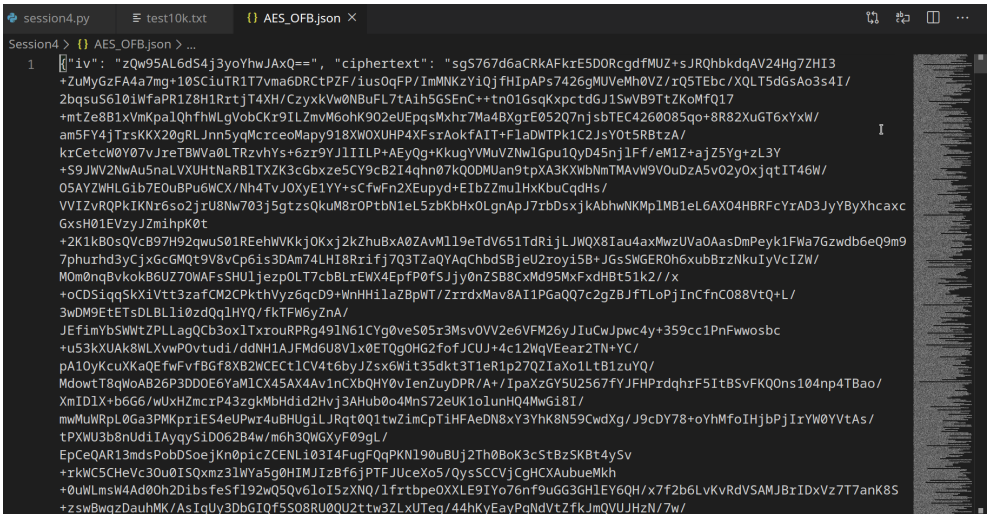

```
key = get_random_bytes(16)
cipher = AES.new(key, AES.MODE_CFB)
ct_bytes = cipher.encrypt( text )
iv = b64encode(cipher.iv).decode('utf-8')
ct = b64encode(ct_bytes).decode('utf-8')
result = json.dumps({'iv':iv, 'ciphertext':ct})

file = open('AES_CFB.json','w')
file.write( result )
file.close()

file = open( 'AES_CFB.json','r' )
json_input = file.read()
file.close()

try:
    b64 = json.loads( json_input )
    iv = b64decode( b64['iv'] )
    ct = b64decode( b64['ciphertext'] )
    cipher = AES.new( key, AES.MODE_CFB, iv=iv )
    pt = cipher.decrypt( ct )
    file = open('Decrypt_AES_CFB.txt','w')
    file.write( pt.decode('UTF-8') )
    file.close()
except ValueError:
    print("Incorrect decryption CFB")
```

7.6 OFB




```
key = get_random_bytes(16)
cipher = AES.new(key, AES.MODE_OFB)
ct_bytes = cipher.encrypt(text)
iv = b64encode(cipher.iv).decode('utf-8')
ct = b64encode(ct_bytes).decode('utf-8')
result = json.dumps({'iv':iv, 'ciphertext':ct})

file = open('AES_OFB.json','w')
file.write( result )
file.close()

file = open( 'AES_OFB.json','r' )
json_input = file.read()
file.close()

try:
    b64 = json.loads(json_input)
    iv = b64decode(b64['iv'])
```

```
    ct = b64decode(b64['ciphertext'])
    cipher = AES.new(key, AES.MODE_OFB, iv=iv)
    pt = cipher.decrypt(ct)
    file = open('Decrypt_AES_OFB.txt','w')
    file.write( pt.decode('UTF-8') )
    file.close()
except ValueError:
    print("Incorrect decryption OFB")
```